CAUL AI — Deal Flow OS Consumer Spec (pre-specified for Phase 1)

Status: SPECIFICATION ONLY — not implemented. No live CAUL AI code is changed by this document. Once you approve Phase 1 **and** provide `DEALFLOW_API_BASE_URL` + `DEALFLOW_API_KEY`, this is a fast, pre-scoped change on the CAUL AI side. Everything stays **staging-only, read-only, aggregate-only**, behind the existing emergency kill switch, and must preserve the 100% coverage gate.

---

1. Secrets / env (CAUL AI) — names the connector will read

Added to `backend/.env` (and CAUL AI deployment Secrets); **no defaults** so missing config fails fast:

```
DEALFLOW_API_BASE_URL   = <Deal Flow OS staging base URL>
DEALFLOW_API_KEY        = <raw read-only key from Deal Flow OS, delivered securely once>
DEALFLOW_TENANT         = dealflowos        # expected tenant; validated on every fetch
```

The connector reports **"Not Connected"** until all three are present (same labeling rule as Phase 1A).

2. RBAC additions (`app/rbac_matrix.py`)

- New permission: `metrics:read` -> "Read Deal Flow OS aggregate growth metrics".
- Granted to: `ExecOwner` (auto via ALL_PERMS), `Admin`, `Analyst`, `Marketing Manager`. (Security/Compliance optional.)
- New **service identity** role `DealFlow Metrics Reader` holding **only** `{metrics:read}`.
- `metrics:read` added to `SERVICE_IDENTITY`-allowed set; it is read-only so it does **not** go in `SERVICE_IDENTITY_FORBIDDEN`.
- CI `test_route_coverage.py` stays green because the new route (`GET /api/metrics/dealflow`) declares `metrics:read`.

3. Connector (`app/connectors/dealflow.py`, new) — implements `BaseConnector`, PULL/read-only

Reuses the existing contract + `run_contract_suite()` gate. External I/O is injected so it stays testable/mPockable.

```
class DealFlowMetricsConnector(BaseConnector):
   name = "dealflow"
   required_permission = "metrics:read"

   def __init__(self, base_url, api_key, tenant, http=None):
       self._cfg = (base_url, api_key, tenant)
       self._http = http or httpx.Client(timeout=5.0)   # timeout bounded
       self._paused = False; self._revoked = False
       self._last_as_of = None; self._audit = []

   def authorize(self, granted):                 # default-deny
       if self.required_permission not in granted:
           raise ConnectorUnauthorized

   def execute(self, action):                    # action.name == "fetch_summary"
       if self._revoked: raise ConnectorRevoked
       if self._paused:  raise ConnectorPaused
       try:
           r = self._http.get(f"{base}/v1/metrics/summary",
                              headers={"Authorization": f"Bearer {key}"})
       except httpx.TimeoutException:
           self._record("timeout"); raise ConnectorTimeout       # no side effect
       if r.status_code != 200:
           self._record("failure"); raise ConnectorFailure       # no side effect
       data = _validate_aggregate_only(r.json(), expected_tenant=tenant)  # reject PII/row-level
       if data["as_of"] == self._last_as_of:
           return ConnectorResult("replayed", "no-op", idempotent_replay=True)  # idempotent
       self._last_as_of = data["as_of"]; self._record("execute")
       return ConnectorResult("executed", "fetched")
```

```
  def pause(self): self._paused = True
  def resume(self): self._paused = False
  def revoke(self): self._revoked = True
  def audit_records(self): return list(self._audit)
```

`_validate_aggregate_only()` enforces an **allow-list** of numeric/aggregate keys and **rejects** any unexpected or row-level/PII-looking field (hard fail) — defense against the source over-sharing.

4. Storage (`app/models.py` + Alembic migration, new)

`dealflow_metrics`: `id`, `tenant`, `as_of` (unique per tenant), `payload` (JSONB aggregates), `source_hash` (sha256 of canonical payload), `fetched_at`. Append-only in practice (upsert by `as_of`).

5. Scheduled pull + kill switch (reuse existing patterns)

- New secured endpoint `POST /api/cron/pull-dealflow-metrics` — same per-env HMAC bearer auth as the existing audit-verify cron (`WEBHOOK_CRON_SECRET_<ENV>`).
- New `.emergent/crons.yml` entry (e.g. hourly) calling that endpoint.
- Before each pull: `active_pause_blocking(db, "connector:dealflow")` -> if paused, **skip** (kill switch).
- Operators pause/resume via the existing Emergency page (scope `granular`, target `connector:dealflow`).
- Every fetch writes an audit event: `action="connector.dealflow.fetch"`, `resource="metrics"`.

6. Read API + UI

- `GET /api/metrics/dealflow` -> latest stored aggregates + `as_of` + freshness state (`connected` / `stale` / `not_connected`); gated by `metrics:read`.
- Dashboard: a "Deal Flow OS growth" card (values + `as_of`, with Not-Connected/Stale states).
- Connectors page: shows `dealflow` connector status (Not Connected -> Connected (staging)).

7. Files that change in Phase 1 (checklist)

- `app/rbac_matrix.py` (perm + role grants)
- `app/connectors/dealflow.py` (new) · `app/connectors/__init__.py` (register)
- `app/models.py` (+`DealFlowMetric`) · new Alembic migration
- `app/routers/metrics.py` (new `GET`) · register in `server.py`
- `app/routers/cron.py` (+pull endpoint) · `.emergent/crons.yml` (+schedule)
- `app/config.py` (`get_dealflow_config()` — fail-fast, no defaults)
- `app/seed.py` (service role, connector registry/settings entry)
- `frontend/src/views/Dashboard.jsx` + `Connectors.jsx` (card + status)
- `backend/tests/` (see §8) — **must keep 100% coverage gate green**

8. Test plan (maps to readiness §5; all with mocked HTTP + synthetic data)

Contract (`run_contract_suite` with a mock transport) · authN (valid/invalid/expired key) · authZ default-deny (`metrics:read`) · tenant-isolation (reject wrong `tenant`) · schema/aggregate-only validation (reject injected PII) · reconciliation (stored == source) · timeout/failure -> no side effect · idempotent replay by `as_of` · kill-switch skip when paused · cron auth (per-env HMAC) · read endpoint RBAC. New lines covered to **100%**.

9. Guardrails preserved

Staging-only · read-only · aggregate-only · least-privilege service identity · bounded timeout/retries/circuit-break · audit every fetch · emergency kill switch · Deal Flow OS has **zero** runtime dependency on CAUL AI · no external spend without separate approval · auth remains the labeled stub (no production IdP change).

Reminder: implementing this is **Phase 1 (staging)** and requires your written go-ahead + the two secrets. This document only pre-specifies it so the change is fast and low-risk when approved.